

NICLabs Chile

COMPARACIÓN DE
ALGORITMOS DE PARTICIÓN DE
GRAFOS: METIS vs.
SPARTSIM

Practicante:
Valeria Serrano
Supervisor de práctica:
Pablo Villar

Enero 2025

Índice de contenidos

1	Introducción	2
2	Metodología	3
2.1	Generación y preprocesamiento de los grafos	3
2.2	Ejecución de los algoritmos de partición y recolección de datos .	3
2.3	Condiciones de ejecución	4
3	Resultados	5
4	Análisis y discusión	12
4.1	Calidad de las particiones	12
4.1.1	ΔV :	12
4.1.2	$\max\{\delta_i\}$:	12
4.2	Tiempo de ejecución	12
4.3	Escalabilidad	13
5	Conclusión	14
6	Posibles mejoras y trabajo futuro	15

1 Introducción

El problema de partición de grafos es un problema relevante hoy en día en el área de la computación, lo que ha llevado al desarrollo de diferentes algoritmos que buscan resolver esta tarea intentando optimizar distintos aspectos.

El presente informe muestra los resultados obtenidos a partir de un experimento de comparación de los algoritmos METIS y SParTSim. Este experimento tuvo por objetivo evaluar el desempeño de ambos algoritmos al particionar grafos.

Durante este experimento se aplicó tanto METIS como SParTSim en distintos grafos con distintas cantidades de nodos y aristas, y se utilizaron distintas métricas para medir aspectos como la calidad de las particiones, el tiempo de ejecución y la escalabilidad, entre otros.

Este informe detalla tanto la metodología empleada en el experimento, como los resultados obtenidos y su respectivo análisis, además de posibles mejoras futuras al experimento mismo.

2 Metodología

La realización de este experimento incluye distintas etapas, la generación de grafos y su preprocesamiento, además de la ejecución de los algoritmos y posterior recopilación de métricas relevantes. Además, fue necesario estudiar algunos papers para entender el funcionamiento de ambos algoritmos, y se utilizó un repositorio preexistente para realizar las particiones.

Cabe señalar que durante el estudio de los papers relacionados a cada algoritmo, a priori, debido a la naturaleza de ambos algoritmos, la hipótesis establecida fue que SParTSim era mejor que METIS para grafos pequeños, mientras que METIS es preferible para grafos más grandes. Esta hipótesis cobra sentido al ver la sección de resultados, y se discute con más calma en la sección de análisis y discusión.

2.1 Generación y preprocesamiento de los grafos

Para poder medir correctamente el rendimiento de los algoritmos, primero fue necesario generar distintos grafos variando la cantidad de nodos y de aristas. Para esto fue necesario implementar un script en Python que permitiera crear grafos aleatorios con una cantidad de nodos y aristas dadas en formato GeoJSON.

Para la realización de este script primero fue necesario revisar la estructura de los grafos en GeoJSON disponibles en el repositorio del programa que particiona los grafos. Se escogió utilizar Python como lenguaje debido a la existencia de su módulo json.

Posterior a generar los grafos con el script, fue necesario preprocesar los grafos formateándolos para obtener la indentación y espaciado específico que lee el programa de particionamiento, ya que el programa presentaba problemas al leer archivos con la indentación usual de GeoJSON.

2.2 Ejecución de los algoritmos de partición y recolección de datos

Una vez generados y formateados los grafos necesarios se procedió a particionarlos con el programa preexistente en el repositorio. Se particionó el mismo grafo reiteradas veces, variando además la cantidad de particiones. Además, fue necesaria la implementación de un tercer script de Python que procesara el grafo original y sus particiones para obtener las distintas métricas a estudiar.

Las métricas que se establecieron para evaluar el desempeño de METIS Y SParTSim son las siguientes:

- Tiempo de ejecución medido en ms: entregado por el programa de particionamiento de grafos para cada algoritmo, los datos se exportaron como

CSV para cada grafo particionado.

- Calidad de la partición: se evalúa teniendo en cuenta el balance de las particiones y la minimización de la cantidad de aristas en el corte:

- Balance de la partición: para medir el balance de la partición se define

$$\Delta V = V_{max} - V_{min}$$

donde

$$* V_{max} = \max\{|V_i| \mid V_i \subseteq V\}$$

$$* V_{min} = \min\{|V_i| \mid V_i \subseteq V\}$$

$$V_i, V_j \subseteq V, V_i \cap V_j = \emptyset, \forall i, j \in \{1, \dots, k\}, i \neq j$$

Donde mientras menor sea el ΔV mejor será la calidad de la partición.

- Minimización de cortes: para esto se comparará el $\max\{\delta_i\}$ de ambos algoritmos, donde se define $\delta_i := E[V_i, V \setminus V_i]$
- Escalabilidad: comportamiento de los algoritmos en grafos con distinta cantidad de nodos y aristas.

2.3 Condiciones de ejecución

- Especificaciones del hardware:
 - CPU: 12th Gen Intel(R) Core(TM) i7-12650H 2.30 GHz
 - RAM: 16GB
 - Sistema Operativo: Windows 11

Para la realización del experimento se generaron distintos grafos de prueba a partir del archivo `generate_geojson.py` y formateados con `format.py` para poder ser procesados por el programa de particionamiento. Se utilizaron las siguientes cantidades de nodos y aristas:

- 1000 nodos: 2000, 4000, 8000, 12000 y 16000 aristas.
- 3000 nodos: 6000, 12000, 18000 y 24000 aristas.
- 5000 nodos: 10000 y 20000 aristas.

Todos estos grafos fueron trabajados utilizando particiones de 4, 8 y 16, y se recopilaban datos como el tiempo en *ms*, utilizando las estadísticas del mismo programa de partición, y adicional a esto, se utilizó el script `partitions_quality.py` para obtener información sobre las métricas de calidad de particiones mencionadas previamente.

3 Resultados

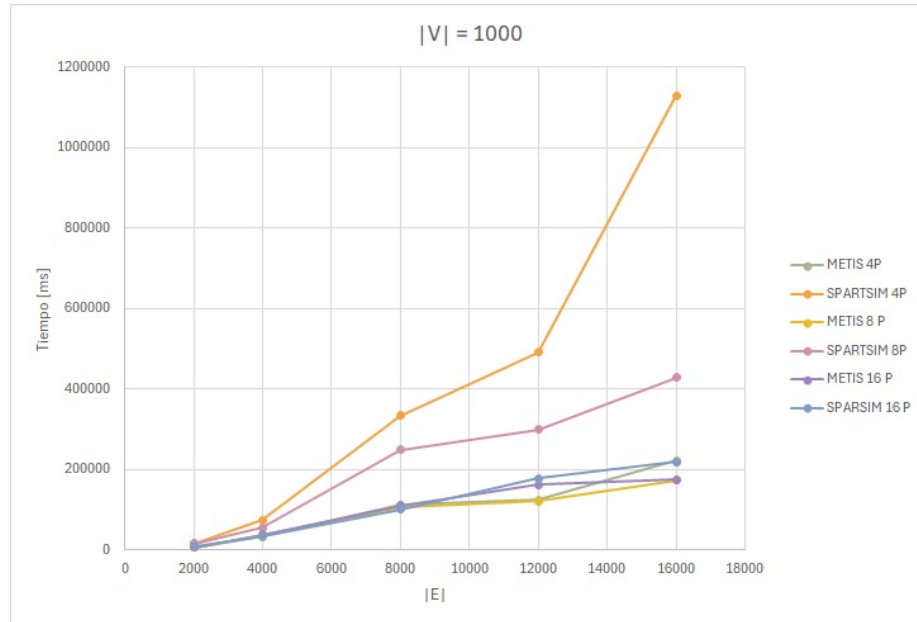


Figure 1: Tiempo de ejecución en *ms* para grafos de 1000 nodos con distinta cantidad de aristas para SPaRTSim y METIS.

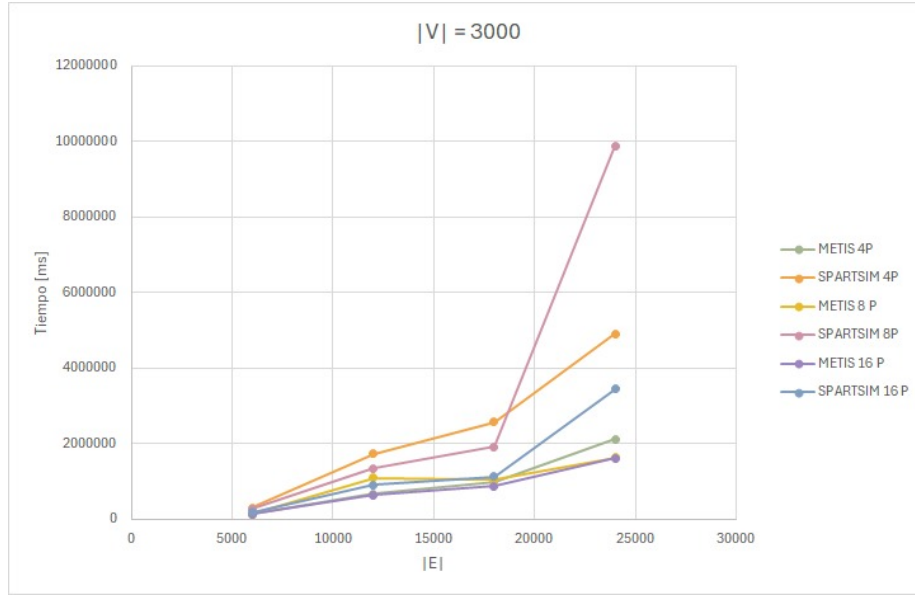


Figure 2: Tiempo de ejecución en *ms* para grafos de 3000 nodos con distinta cantidad de aristas para SPArTSim y METIS.

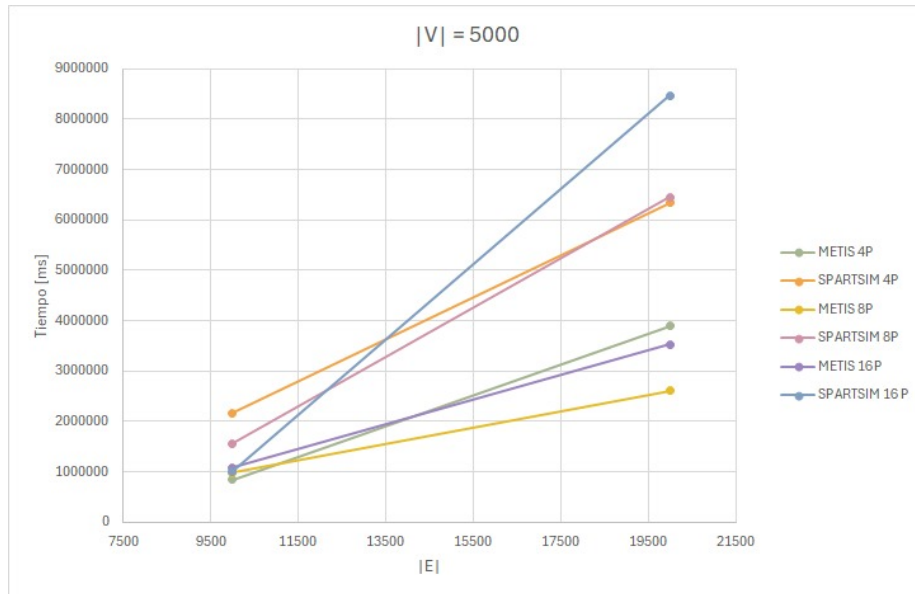


Figure 3: Tiempo de ejecución en *ms* para grafos de 5000 nodos con distinta cantidad de aristas para SPArTSim y METIS.

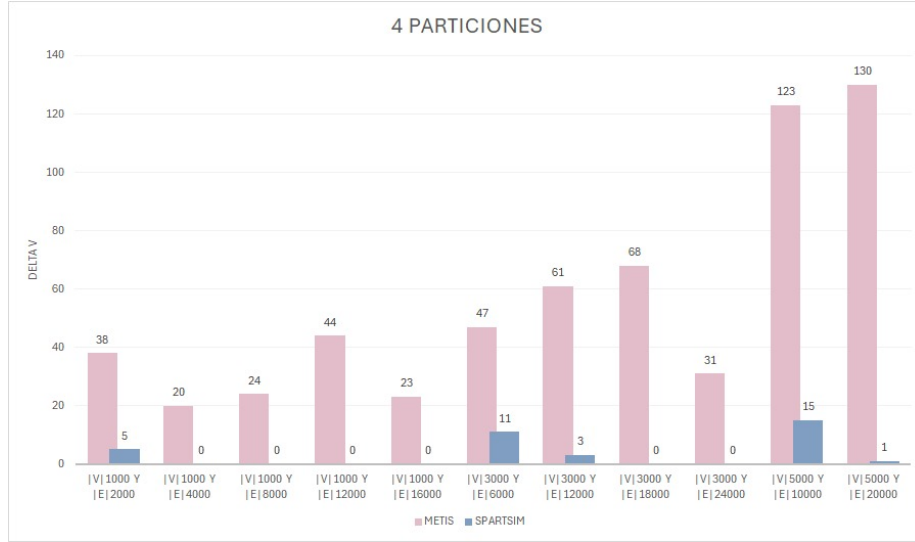


Figure 4: ΔV para grafos con 4 particiones.

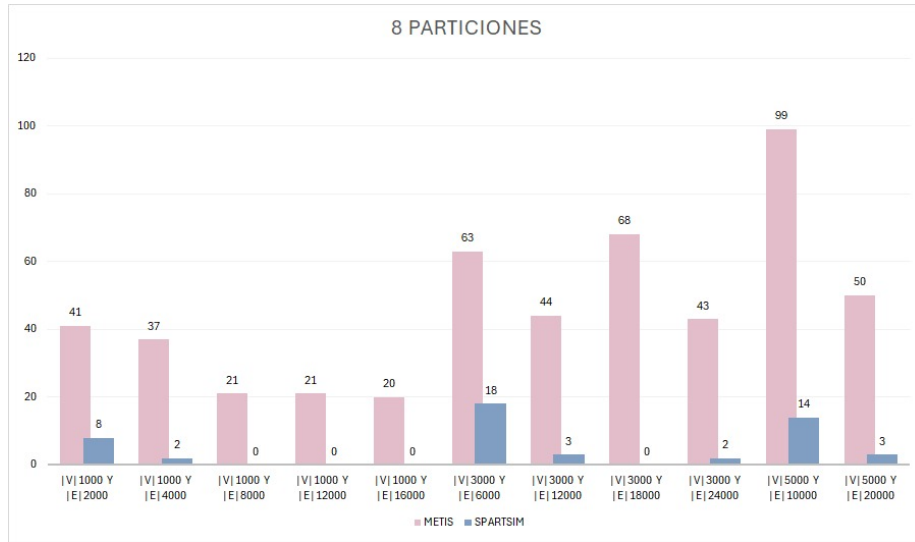


Figure 5: ΔV para grafos con 8 particiones.

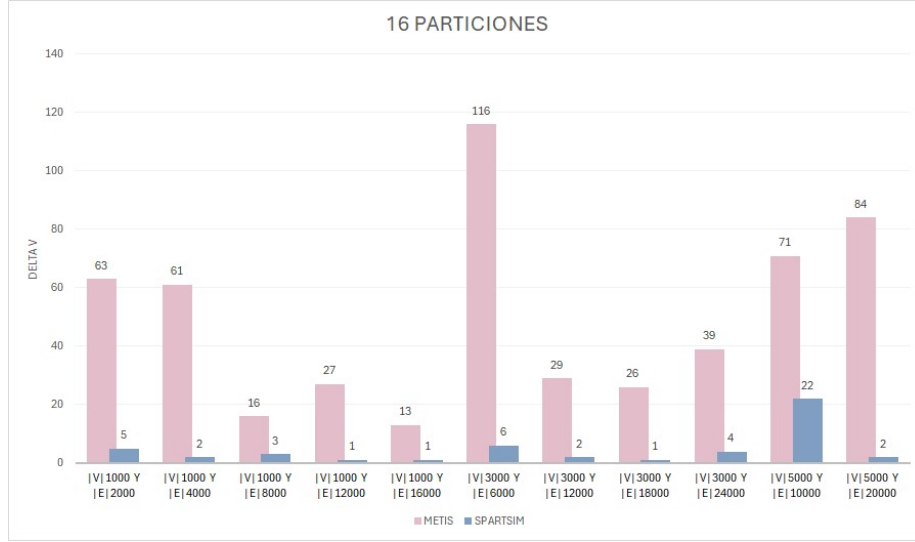


Figure 6: ΔV para grafos con 16 particiones.

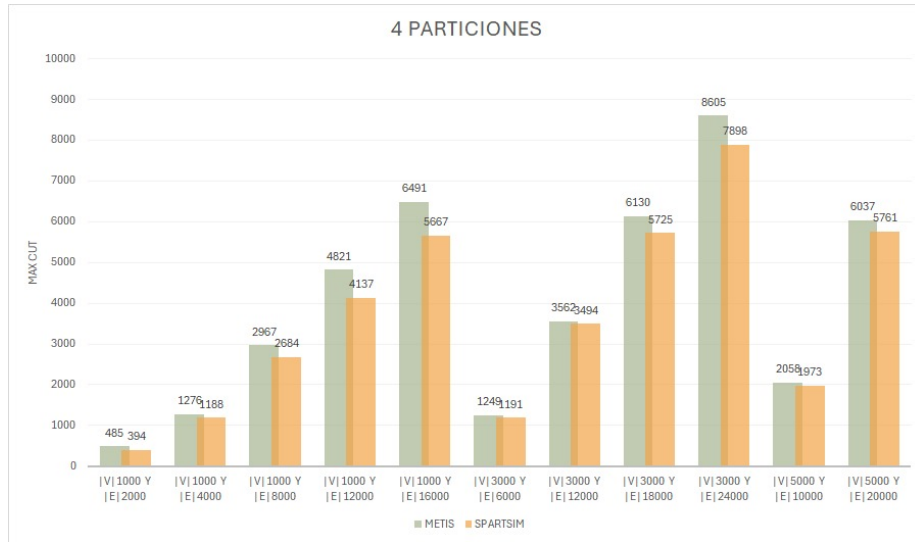


Figure 7: $\max\{\delta_i\}$ para grafos con 4 particiones.

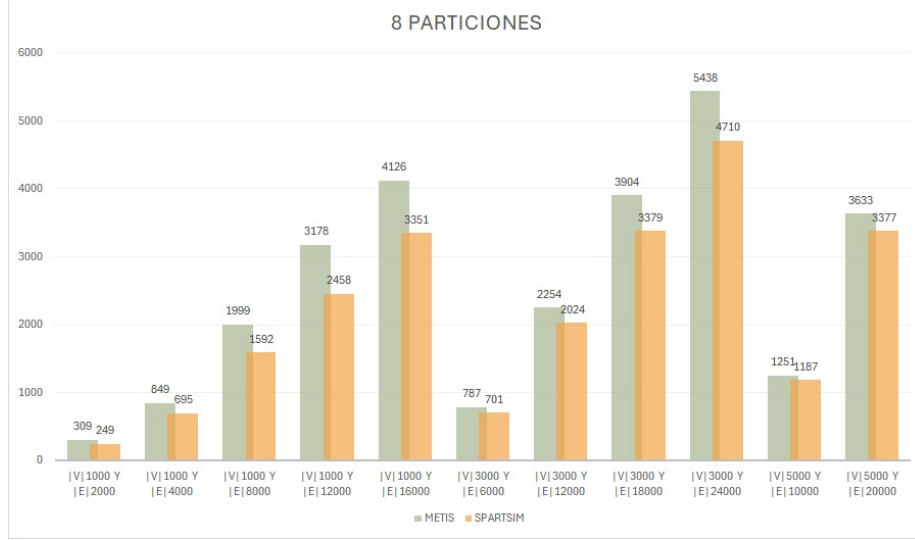


Figure 8: $\max\{\delta_i\}$ para grafos con 8 particiones.

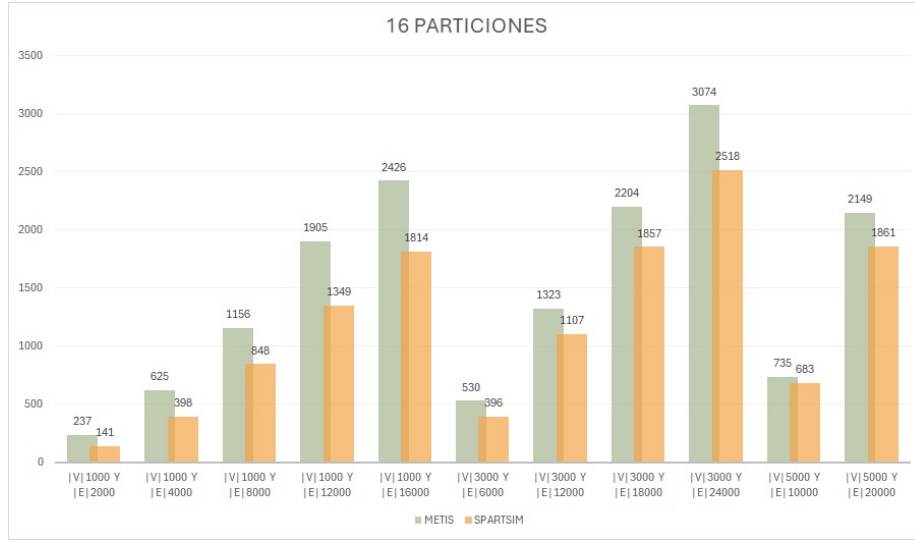


Figure 9: $\max\{\delta_i\}$ para grafos con 16 particiones.

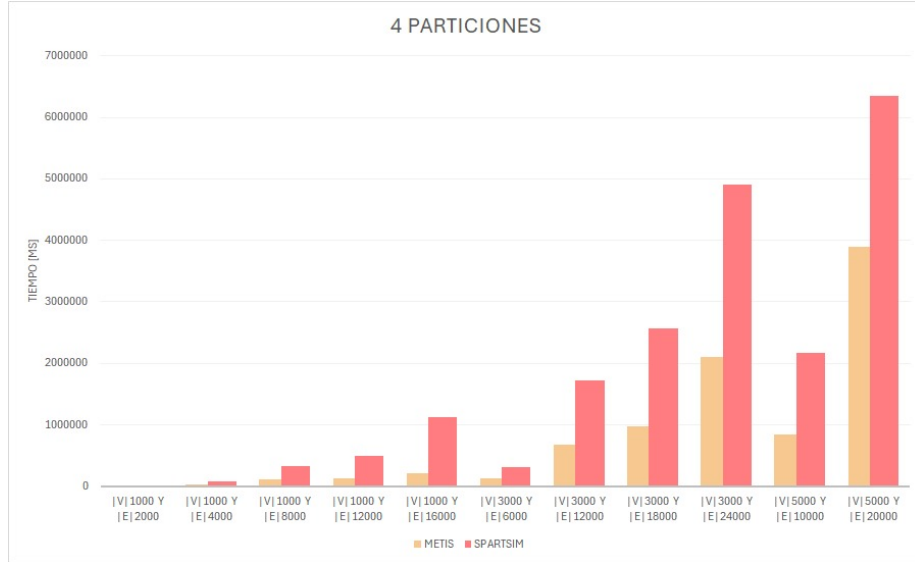


Figure 10: Tiempo en *ms* para grafos con 4 particiones.

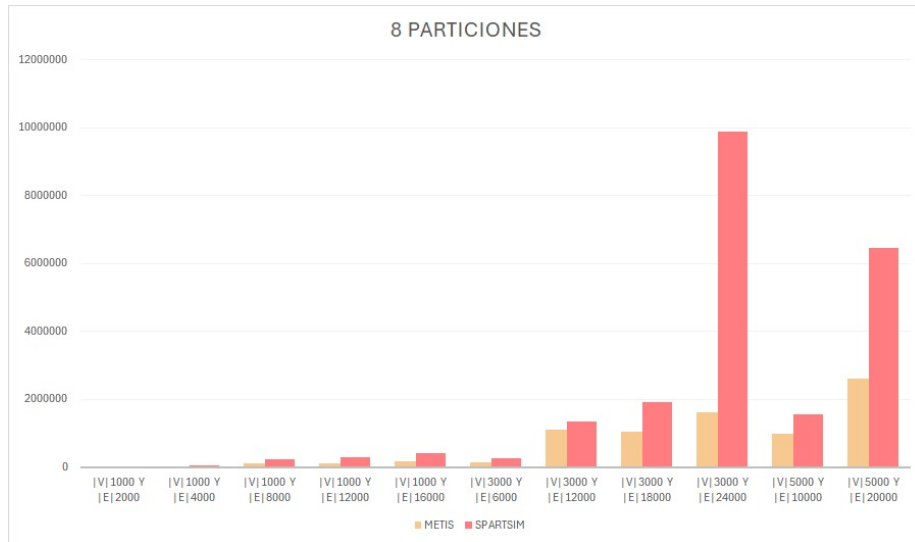


Figure 11: Tiempo en *ms* para grafos con 8 particiones.



Figure 12: Tiempo en *ms* para grafos con 16 particiones.

4 Análisis y discusión

4.1 Calidad de las particiones

4.1.1 ΔV :

- METIS muestra valores de ΔV superiores en todos los casos testeados, lo que indica que, en general, su particionamiento es de peor calidad que SPaRTSim, dado que sus particiones no son lo suficientemente balanceadas en comparación y se podrían mejorar.
- Esto sugiere que SPaRTSim es un mejor algoritmo para conseguir k-way partitions en comparación a METIS.
- SPaRTSim en general presenta valores de ΔV considerablemente más bajos, donde su valor máximo fue de 22 para el grafo de 5000 nodos con 10000 aristas y una partición de 16, mientras que el valor máximo alcanzado por METIS fue de 130 para el grafo de 5000 nodos y 20000 aristas con una partición de 4.

4.1.2 $\max\{\delta_i\}$:

- METIS también tiende a tener valores más altos como su corte máximo en comparación a SPaRTSim. Un corte mayor generalmente implica que el algoritmo obtiene particiones que maximizan la interacción entre ellas, lo cual, en el caso del problema de minimización de cortes, no resulta tan beneficioso.
- SPaRTSim presenta cortes más pequeños, lo que sugiere que sus particiones son de mejor calidad, teniendo en cuenta además que presenta particiones considerablemente más balanceadas.

4.2 Tiempo de ejecución

- El tiempo de ejecución de SPaRTSim es significativamente mayor que el de METIS. En la mayoría de los casos, SPaRTSim presenta entre el doble y el triple de tiempo de ejecución en comparación a METIS.
- Este aumento en el tiempo puede ser explicado por la diferencia entre un algoritmo offline y uno online. METIS, al ser un algoritmo offline, tiene disponible la información completa del grafo desde el comienzo, en contraste a SPaRTSim, que es un algoritmo online, por lo que se va actualizando a medida que va recibiendo más información. Con esto en cuenta, una buena hipótesis es que SPaRTSim tiene un mayor tiempo de ejecución comparado a METIS, ya que mientras METIS sacrifica la calidad de sus particiones para obtener un mejor tiempo de ejecución, SPaRTSim prioriza la calidad de las particiones aunque eso implique un mayor costo a nivel computacional.

4.3 Escalabilidad

- METIS parece tener una escalabilidad más eficiente, ya que a medida que aumenta la cantidad de nodos y aristas (grafos más grandes), el tiempo de ejecución de METIS aumenta en menor medida que SParTSim. Esto sugiere que METIS resulta más útil para trabajar con grafos de mayor tamaño, donde además la calidad de las particiones podría pasar más desapercibida (aunque sería necesario ampliar el muestreo del experimento para confirmar esto, pero a priori es una buena hipótesis).
- SParTSim, por otro lado, muestra un incremento en el tiempo de ejecución muchísimo más pronunciado conforme crece el número de nodos y aristas. Esto sugiere que SParTSim no escala tan bien para grafos demasiado grandes. A medida que el tamaño de un grafo aumenta, su complejidad computacional podría hacer que se vuelva ineficiente el uso de este algoritmo, pese a la buena calidad de sus particiones.

5 Conclusión

- METIS es más eficiente en términos de tiempo de ejecución, pero obtiene peores resultados en cuanto a la calidad de las particiones, tanto para el ΔV como para el $\max\{\delta_i\}$, lo que lo convierte en una opción menos atractiva para particionar grafos más pequeños, pero sí puede resultar una mejor elección si se trata de grafos de mayor tamaño.
- SParTSim, aunque ofrece una calidad de partición considerablemente mejor que METIS, no es tan eficiente para utilizarse en grafos de gran tamaño, por lo que es preferible utilizarlo para grafos de menor tamaño y menor densidad.
- Si se busca escalabilidad y eficiencia en tiempo, METIS es claramente el algoritmo preferido entre ambos, mientras que SParTSim podría resultar útil en contextos donde la equidad en el particionamiento y la calidad de este sea más importante que el tiempo de ejecución, aunque con la limitante de que el tiempo de ejecución vuelve a SParTSim una opción mucho menos atractiva.

6 Posibles mejoras y trabajo futuro

Dentro de las posibles mejoras que se proponen para este experimento se encuentran:

- Mejora del script `generate_geojson.py`: se propone utilizar BFS o DFS para generar un spanning tree y asegurar que la creación de grafos con misma cantidad de aristas y nodos tome en cuenta la cantidad pedida de nodos dentro de una única componente conexa, ya que el script actual solo toma nodos al azar dentro de la lista con la cantidad de nodos generada, lo que no garantiza que se usen todos los nodos en el grafo conexo resultante.
- Se propone pasar los scripts de Python a Julia, debido a que Julia es un lenguaje especialmente pensado para trabajar con grandes cantidades de datos, lo que puede resultar útil en caso de querer trabajar con grafos mucho más grandes y densos en comparación a los utilizados en este experimento. Además, Julia cuenta ya con un módulo para trabajar archivos de tipo `json`.
- Otra de las mejoras que se propone al script `generate_geojson.py` es hacer que los archivos generados tengan ordenados de forma ascendente sus `init_node` y `term_node`, ya que está la hipótesis de que en el programa de partición de grafos, esto ayudaría a minimizar el tiempo de particionamiento de los algoritmos, dada la forma en que está codificado el programa.

Dentro de los comentarios con respecto al experimento, algo bastante curioso con respecto al programa de particionamiento es que si se particiona con SParTSim un grafo con una partición pequeña, por ejemplo, 4, entonces al aumentar la cantidad de particiones, por ejemplo, a 8 particiones, el tiempo de particionamiento disminuye comparado a particionar el mismo grafo en 8 desde el comienzo.

Otra cosa que vale la pena mencionar, también con respecto a SParTSim, es que siempre que se particiona el mismo grafo en la misma cantidad de partes, el resultado de SParTSim será el mismo, en comparación a METIS, que en cada intento entrega nuevos valores para el ΔV y $\max\{\delta_i\}$. Esto sugiere que SParTSim podría estar llegando siempre a la partición más óptima en cuando a balanceo y minimización de corte de las particiones.